What is Oracle?

Oracle is an American information technology (IT) company that offers a variety of products and services. Oracle is known for its database management systems (DBMS), cloud applications, and cloud infrastructure.

- **Oracle was founded in California in 1977 by Larry Ellison, Bob Miner, and Ed Oates**
- **The name Oracle comes from the code name of a CIA-funded project Ellison worked on while employed by Ampex.**

What is a Database?

A database refers to the **organized collection of structured data** stored electronically in a device. It allows us to access, manage, and find relevant information frequently. The flat file structure was extensively used to store data before the database system was invented. The relational database approach becomes popular in comparison to the flat file model because it eliminates redundant data. **For example**, suppose we have an employee and contact information stored in the same file. In such a case, the employees with multiple contacts will show up in many rows.

The RDBMS system manages the relational data. Oracle Database is the most famous relational database system (RDBMS) because it shares the largest part of a market among other relational databases. Some other popular relational databases are MySQL, DB2, SQL Server, PostgreSQL, etc.

What is the Oracle database?

Oracle database is a relational database management system (RDBMS) from Oracle Corporation. Oracle Database is a database management system (DBMS) that stores and manages data for organizations. It a relational database management system (RDBMS) that uses structured query language (SQL) to manipulate and query data.

Oracle database is a relational database management system. It is also called **OracleDB**, or simply **Oracle**. It is produced and marketed by **Oracle Corporation**. It was created in **1977** by **Lawrence Ellison** and other engineers. It is one of the most popular relational database engines in the IT market for storing, organizing, and retrieving data.

The following is a list of Oracle database editions in order of priority:

- **Enterprise Edition:** It is the most robust and secure edition. It offers all features, including superior performance and security.
- **Standard Edition:** It provides the base functionality for users that do not require Enterprise Edition's robust package.
- **Express Edition (XE):** It is the lightweight, free and limited Windows, and Linux edition.
- **Oracle Lite:** It is designed for mobile devices.
- **Personal Edition:** It's comparable to the Enterprise Edition but without the Oracle Real Application Clusters feature.

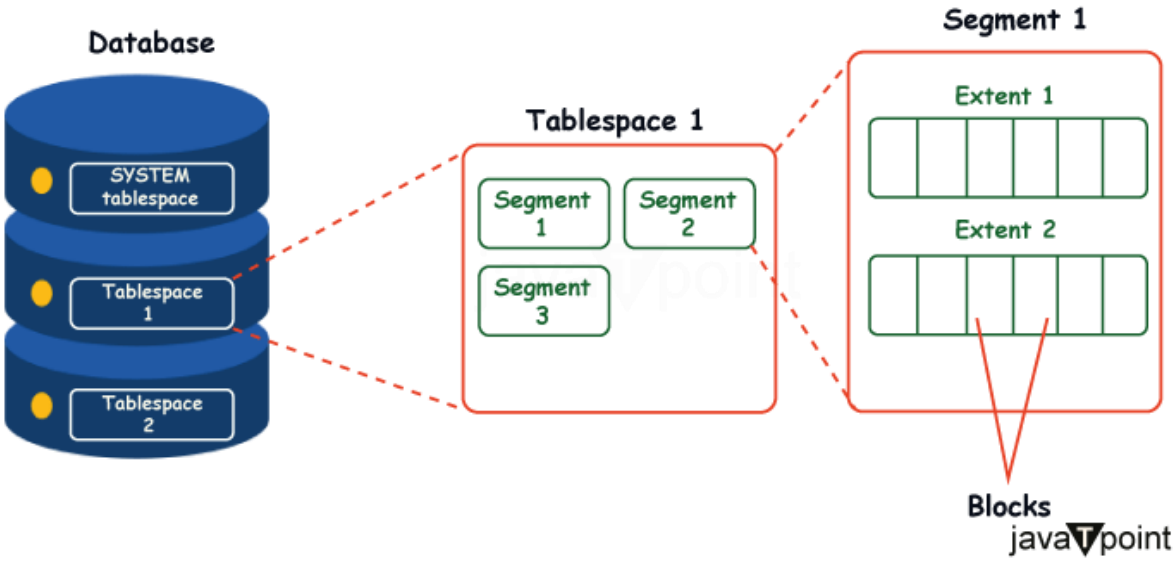
Oracle Database Architecture

Introduction

- Oracle Database is a widely used relational database management system (RDBMS) developed by Oracle Corporation. It is known for its robustness, scalability, and comprehensive feature set. Understanding the architecture of Oracle Database is essential for efficient management and utilization of the system.
- The architecture of Oracle Database can be divided into two main components: the logical and physical structures.

Logical Structure:

- **Logical Structure:** It represents the components that you can see in an Oracle database such as tables, views, indexes etc. In this structure, Oracle divides the database into smaller units to manage, store and retrieve the database efficiently.

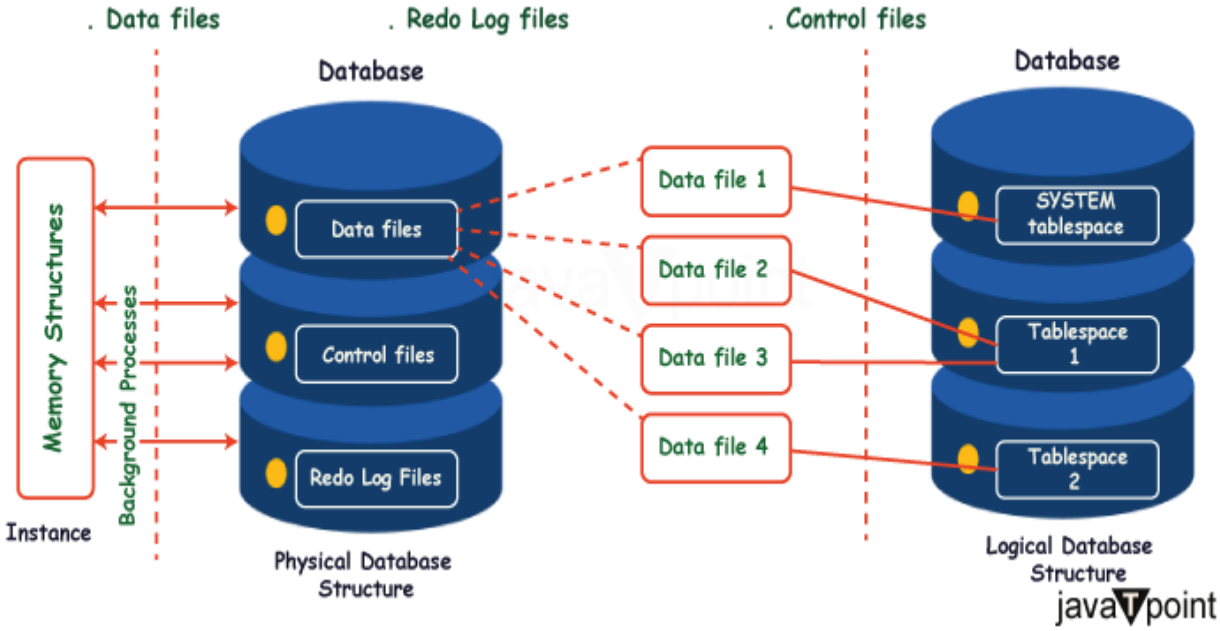


- **Database:** At the highest level, Oracle Database is organized into one or more logical databases, also known as tablespaces. Each tablespace consists of multiple data files that store the actual data.
    - Tablespace is a logical storage unit that groups related logical structures like tables, indexes, and other database objects. It acts as a bridge between the physical and logical storage of data in the database.
    - A **segment** in Oracle Database is a set of storage structures that hold data for specific database objects such as tables, indexes, or partitions. It is a logical construct that Oracle uses to manage and store data at the database level.
    - **EXTEND** for a segment refers to increasing the amount of space allocated to a specific segment, such as a table, index, or rollback segment.
    - When a segment (e.g., a table or an index) grows beyond its initially allocated space, Oracle provides the ability to **extend** that segment, allowing it to use more space in the tablespace.
- **Schema:** A schema is a collection of database objects owned by a specific user. It includes tables, views, indexes, procedures, and other schema objects.
- **Table:** Tables are the fundamental data storage units in the Oracle Database. They hold the actual data in rows and columns.
- **Indexes:** Indexes are optional structures that enhance query performance by allowing quick access to specific data. They are created on one or more columns of a table.
- **Views:** Views are virtual tables derived from the data stored in one or more tables. They provide a customized and simplified view of the data to the users.

Physical Structure:

Physical Structure: It is a collection of operating system files in the database. It consists of three types of physical files.

Following figure represents the relationship between the physical and logical database structure.



- **Instance:** An instance represents the Oracle Database running on a server. It comprises memory structures (System Global Area - SGA) and background processes. The SGA is divided into the database buffer cache, shared pool, redo log buffer, and more.

- **Background Processes:** Oracle Database has multiple background processes that perform tasks such as managing memory, I/O operations, recovery, and maintaining database integrity.

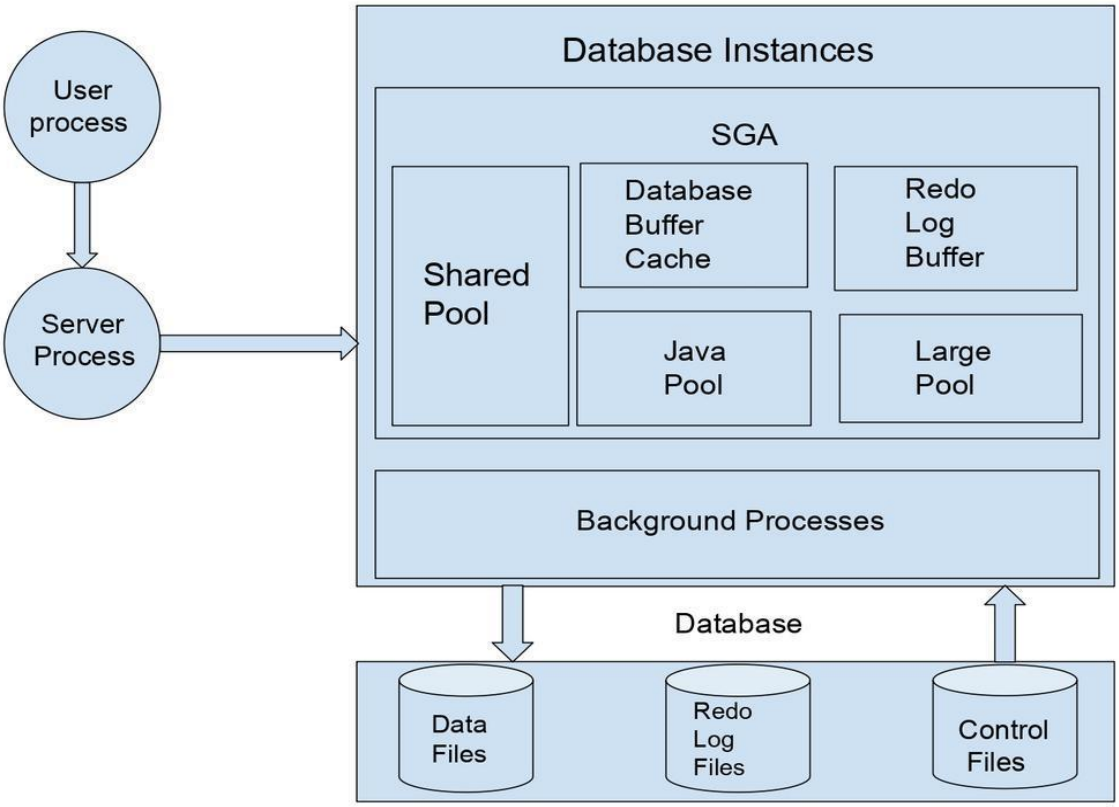
- **Data Files:** Data files are physical files stored on the operating system's file system or storage devices. They store the actual data of the database and are organized into tablespaces. **data file** is a physical file used by the Oracle Database to store data. It is part of the database storage architecture and is used to store data for tables, indexes, and other objects.

- **Control Files:** Control files are binary files that store metadata about the physical structure of the database. They contain information about the database name, data file locations, redo log file locations, and more.

- **Redo Logs:** Redo logs It records all changes made to the database, such as inserts, updates, and deletions, as well as transactions that modify data. Redo log files ensure data integrity and allow for recovery in case of failures

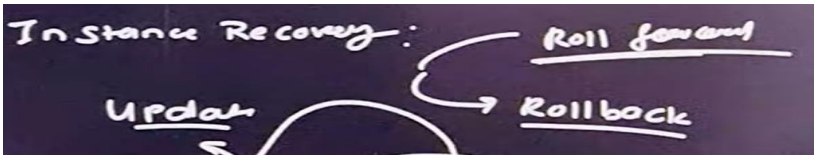
- **Archive Logs:** Archive logs are copies of the redo logs archived and stored for backup and recovery.

Oracle Database follows a client-server architecture, where multiple client applications can connect to a single Oracle Database instance. The clients communicate with the database through Oracle Net Services, which manages network connectivity and authentication.

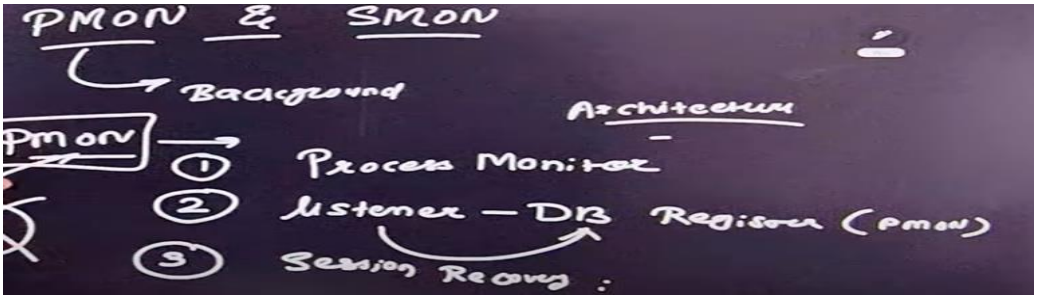


Background processes: Oracle has a collection of processes that are called background processes. These processes are responsible for managing memory, performing I/O operations, and other maintenance activities. Following are some important background processes that are required:

- **System Monitor Process (SMON)**: These processes are responsible for performing system-level recovery and maintenance activities.



- **Process Monitor Process (PMON):** The task of these processes is to monitor other background processes.



- **Database Writer Process (DBWR):** This process performs the task of writing data blocks from the Database Buffer Cache (present in SGA) to physical data files(Present in the Database system).

- **Log Writer Process (LGWR):** This process writes the Redo blocks from Redo Log Buffer (present in SGA) to Redo Log Files(present in the Database system).
- **CheckPoint (CKPT):** This process maintains data files and control files with the most recent checkpoint information.

Oracle database design

Oracle database design is the process of creating a standard for database objects, such as tables, columns, and indexes. The goal of database design is to create physical and logical models of a database system

Why is database design important?

- It helps to reduce errors
- It helps to promote communication
- It helps to make the integration and upgrade processes smoother

Migration and management of Oracle refer to the processes and practices involved in moving Oracle databases, applications, or workloads from one environment to another, as well as managing Oracle systems to ensure their performance, availability, and reliability. Here's an overview of both aspects:

1. Migration of Oracle:

This involves transferring Oracle databases, applications, or infrastructure to a different platform or environment. Common scenarios include:

a. Types of Migration:

- **On-Premises to Cloud:** Migrating Oracle databases and applications from on-premises infrastructure to Oracle Cloud Infrastructure (OCI) or other cloud providers like AWS, Azure, or Google Cloud.
- **Cloud-to-Cloud:** Moving Oracle workloads from one cloud provider to another (e.g., AWS to OCI).
- **Version Upgrades:** Upgrading Oracle database versions or applications to take advantage of new features and improved performance.
- **Platform Migration:** Moving from one operating system or hardware platform to another (e.g., Windows to Linux).

---

b. Tools and Techniques for Migration:

- **Oracle Data Pump:** High-performance data export and import utility for moving data between databases.
- **Oracle GoldenGate:** A real-time data replication tool for minimizing downtime during migration.
- **RMAN (Recovery Manager):** Used for backup-based migrations.
- **Transportable Tablespaces:** Allows tablespaces to be moved between databases.
- **Oracle Cloud Infrastructure Migration Services:** Tools like Oracle Database Migration Service or OCI Block Volume backups to streamline migrations.
- **Third-Party Tools:** Tools like AWS Database Migration Service (DMS), Azure Database Migration Service, or custom scripts.

---

c. Challenges in Migration:

- **Downtime:** Minimizing the impact of migration on end-users.
- **Data Integrity:** Ensuring no data loss or corruption during migration.
- **Compatibility:** Verifying that the new environment supports all features of the Oracle database or application.
- **Performance:** Tuning the database post-migration for optimal performance.
- **Testing:** Extensive pre- and post-migration testing to ensure success.

2. Management of Oracle:

This involves the administration, monitoring, and optimization of Oracle databases and applications to maintain high performance and reliability.

a. Key Management Activities:

- **Database Administration (DBA):**
    - Managing users, roles, and security.
    - Scheduling backups and ensuring disaster recovery readiness.
    - Monitoring and tuning database performance.
- **Patching and Upgrades:** Regularly applying Oracle patches and updates to fix vulnerabilities and improve functionality.
- **Monitoring:** Using tools like Oracle Enterprise Manager (OEM) to monitor database health, performance, and resource utilization.
- **High Availability (HA) and Disaster Recovery (DR):**
    - Setting up Oracle RAC (Real Application Clusters) or Oracle Data Guard for HA/DR.
    - Ensuring continuous availability in case of failure.
- **Cloud Management:** Administering Oracle Cloud Infrastructure (OCI) environments, including cost optimization and scaling.
- **Automation:** Using scripts or tools to automate repetitive DBA tasks like backups, index maintenance, and monitoring.

---

b. Tools for Management:

- **Oracle Enterprise Manager (OEM):** Comprehensive management platform for Oracle environments.
- **SQL Developer:** Tool for database development, debugging, and management.
- **Oracle Autonomous Database:** Reduces administrative overhead by automating patching, tuning, and backup tasks.
- **Third-Party Tools:** Tools like SolarWinds, Nagios, or Quest Foglight for additional monitoring and management capabilities.

---

c. Best Practices for Oracle Management:

- Regularly monitor performance metrics such as CPU, memory, and I/O usage.
- Implement proper backup and recovery strategies.
- Keep Oracle software up to date with patches and upgrades.

- Enforce security best practices, including role-based access control and encryption.
- Conduct regular performance tuning and audits.

Database Schema

A database schema is a structure that represents the logical storage of the data in a database. It represents the organization of data and provides information about the relationships between the tables in each database. In this topic, we will understand more about database schema and its types. A schema is a collection of database objects owned by a user. Essentially it is nothing more than a user that owns objects such as tables, views, etc. These database objects are what make the schema.

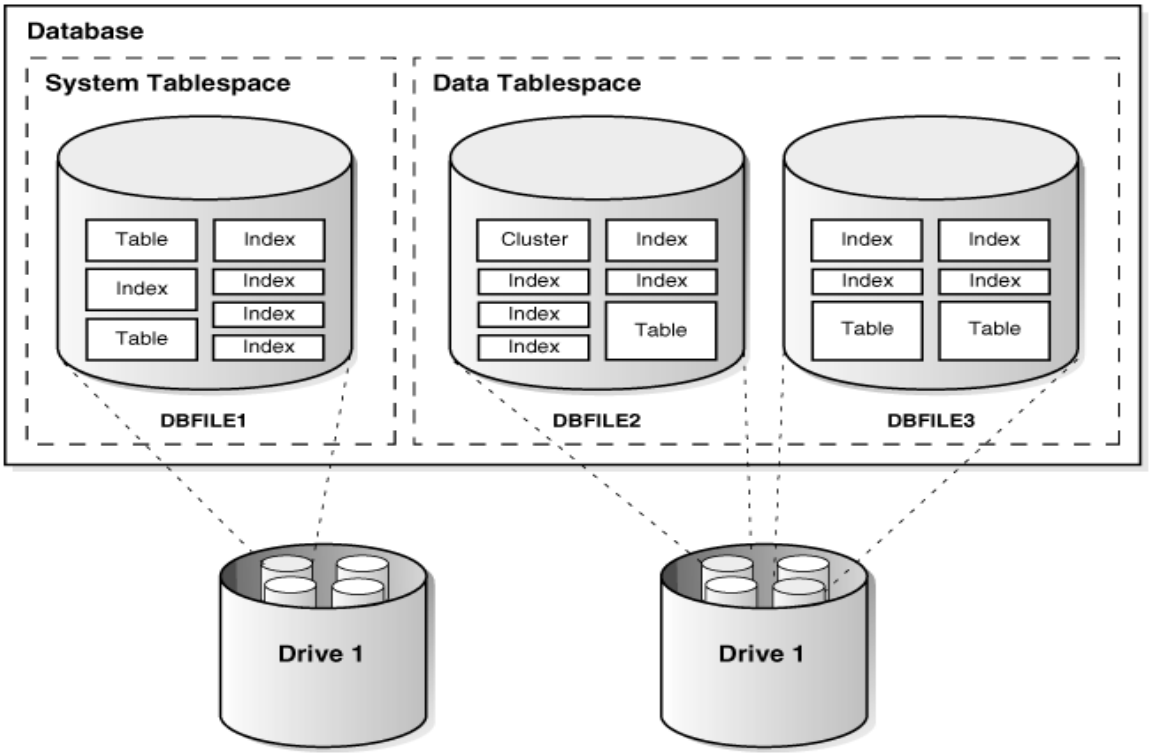
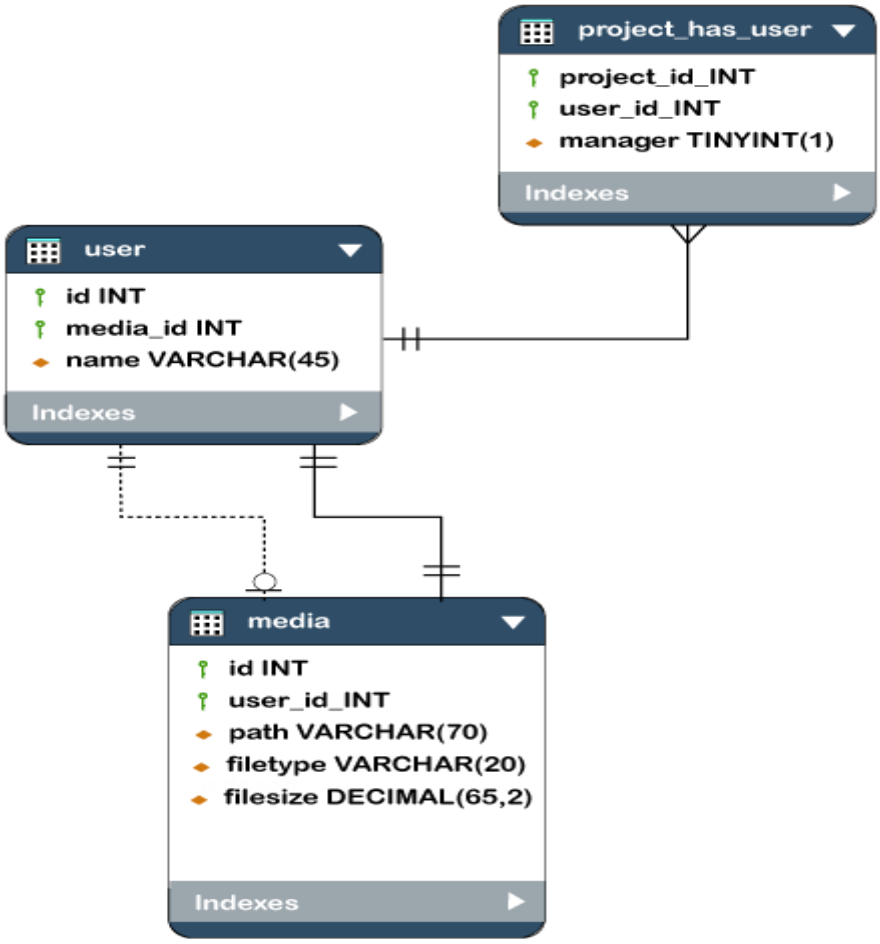


Figure: Illustrates the relationship among objects, tablespaces, and datafiles.

What is Database Schema?

- A database schema is the logical representation of a database, which shows how the data is stored logically in the entire database. It contains a list of attributes and instructions that informs the database engine how the data is organized and how the elements are related to each other.
- A database schema contains schema objects that may include **tables, fields, packages, views, relationships, primary key, foreign key,**
- In actual, the data is physically stored in files that may be in unstructured form, but to retrieve it and use it. we need to put it in a structured form. To do this, a database schema is used. It provides knowledge about how the data is organized in a database and how it is associated with other data.
- **The schema does not physically contain the data itself; instead, it gives information about the shape of data and how it can be related to other tables or models.**
- A database schema object includes the following:
    - Consistent formatting for all data entries.
    - Database objects and unique keys for all data entries.
    - Tables with multiple columns, and each column contains its name and datatype.
- The complexity & the size of the schema vary as per the size of the project. It helps developers to easily manage and structure the database before coding it.

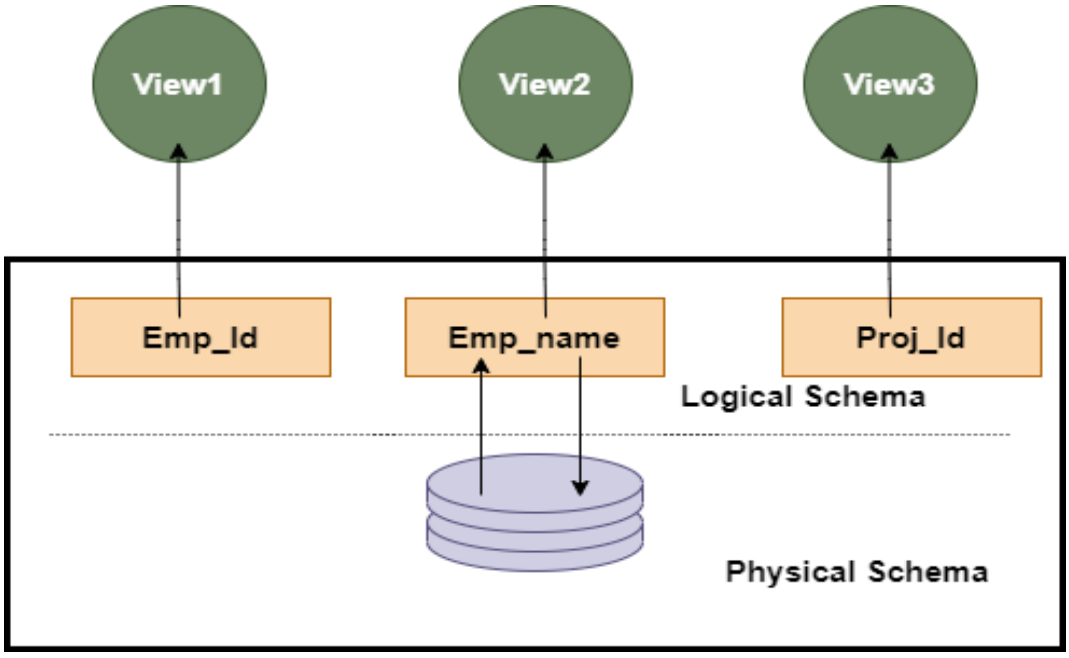
- The given diagram is an example of a database schema. It contains three tables, their data types. This also represents the relationships between the tables and primary keys as well as foreign keys.



Types of Database Schema

The database schema is divided into three types, which are:

1. **Logical Schema**
2. **Physical Schema**
3. **View Schema**



1. Physical Database Schema

A physical database schema specifies how the data is stored physically on a storage system or disk storage in the form of Files and Indices. Designing a database at the physical level is called a **physical schema**.

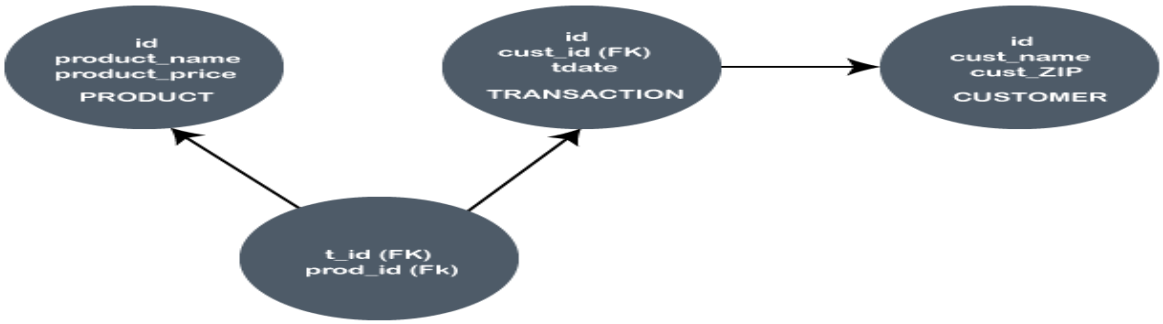
2. Logical Database Schema

The Logical database schema specifies all the logical constraints that need to be applied to the stored data. It defines the views, integrity constraints, and table. Here the term **integrity constraints** define the set of rules that are used by DBMS (Database Management System) to maintain the quality for insertion & update the data. The logical schema represents how the data is stored in the form of tables and how the attributes of a table are linked together.

Various tools are used to create a logical database schema, and these tools demonstrate the relationships between the components of your data; this process is called **ER modelling**.

The ER modelling stands for entity-relationship modelling, which specifies the relationships between different entities.

We can understand it with an example of a basic commerce application. Below is the schema diagram, the simple ER model representing the logical flow of transaction in a commerce application.



In the given example, the Ids are given in each circle, and these Ids are primary key & foreign keys.

3. View Schema

The view level design of a database is known as **view schema**. This schema generally describes the end-user interaction with the database systems.

Data Dictionary Views in Databases

A **data dictionary** in a database is a collection of metadata (data about data). It provides detailed information about database objects, such as tables, columns, indexes, constraints, users, and privileges. The **data dictionary views** allow database administrators and developers to query this metadata.

These views are essential for managing and maintaining the database, offering insights into its structure and security.

Types of Data Dictionary Views

The views in a data dictionary are typically categorized into three levels based on the user's access and scope:

1. **USER Views:**
    - Provide information about objects owned by the current user.
    - Example: USER_TABLES, USER_TAB_COLUMNS.
2. **ALL Views:**

- Provide information about objects accessible to the user, including objects owned by other users that the current user has privileges to access.
- Example: ALL_TABLES, ALL_TAB_COLUMNS.

3. **DBA Views:**
   - Provide a comprehensive view of all objects in the database.
   - Accessible only to users with administrative privileges.
   - Example: DBA_TABLES, DBA_TAB_COLUMNS.

Why Use Data Dictionary Views?

- **Database Management:** Monitor and manage database objects.
- **Debugging and Auditing:** Identify issues, check privileges, or track modifications.
- **Performance Tuning:** Analyse statistics and optimize queries.
- **Security:** Ensure roles and privileges are correctly assigned.

Understanding and leveraging data dictionary views are crucial for effective database administration and development.

Standard Packages in Oracle

Oracle provides **standard packages** to simplify and extend the functionality of the database. These are pre-built PL/SQL packages containing procedures and functions for various operations.

Commonly Used Standard Packages

1. **DBMS_OUTPUT**:
   - Used for debugging by displaying messages.
   - Example:

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello, Oracle!');
END;
```

2. **DBMS_SCHEDULER**:
   - Manages and schedules database jobs.
   - Example:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
    job_name        => 'example_job',
    job_type        => 'PLSQL_BLOCK',
    job_action      => 'BEGIN DBMS_OUTPUT.PUT_LINE("Scheduled Task"); END;',
    start_date      => SYSDATE,
    repeat_interval => 'FREQ=DAILY; BYHOUR=9',
    enabled         => TRUE
  );
END;
```

3. **DBMS_METADATA**:
    - Retrieves metadata about database objects.
    - Example:

```
SELECT DBMS_METADATA.GET_DDL('TABLE', 'EMPLOYEES') FROM DUAL;
```

4. **DBMS_SQL**:
    - Executes dynamic SQL.
    - Example:

```
DECLARE
  cursor_id INTEGER;
BEGIN
  cursor_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(cursor_id, 'SELECT * FROM EMPLOYEES', DBMS_SQL.NATIVE);
  DBMS_SQL.CLOSE_CURSOR(cursor_id);
END;
```

5. **UTL_FILE**:
    - Handles file input and output operations.
    - Example:

```
DECLARE
  file_handle UTL_FILE.FILE_TYPE;
BEGIN
  file_handle := UTL_FILE.FOPEN('DATA_DIR', 'output.txt', 'W');
  UTL_FILE.PUT_LINE(file_handle, 'Hello, File!');
  UTL_FILE.FCLOSE(file_handle);
END;
```

6. **DBMS_LOB**:
    - Handles operations on Large Objects (LOBs), such as BLOBs and CLOBs.
7. **DBMS_STATS**:
    - Gathers and manages optimizer statistics.
8. **DBMS_ALERT**:
    - Implements asynchronous notifications for database events.
9. **DBMS_CRYPTO**:
    - Provides encryption and decryption utilities.
10. **DBMS_LOCK**:
    - Provides locking mechanisms for custom applications.

b. Advantages of Standard Packages

- Simplifies common operations (e.g., debugging, scheduling, metadata retrieval).
- Reduces development time with pre-built utilities.

- Provides a consistent and tested interface for database operations.
- Enhances database management capabilities.

Maintaining the control

"Maintaining control in Oracle" refers to the practice of managing and ensuring the integrity of the Oracle database's "control file," a crucial binary file that stores vital information about the database structure, including the location of data files and redo logs, essentially acting as a central point of reference for the database to function properly, this involves regularly backing up the control file, maintaining multiple copies for redundancy, and monitoring its status to prevent potential data loss or disruption in case of system failures.

Key aspects of maintaining control in Oracle:

- **Multiple control files:**

Oracle recommends having at least two control files on separate disks to prevent data loss if one becomes unavailable.

- **Regular backups:**

Regularly backing up the control file is essential for recovery purposes, as a corrupted or lost control file can significantly impact database accessibility.

- **Monitoring control file status:**

Utilize Oracle's data dictionary views to check the health of the control file, including its location, size, and timestamps.

- **Recovery procedures:**

In case of a control file corruption, Oracle provides mechanisms to recover the control file from a backup copy.

- **ALTER DATABASE commands:**

Use the ALTER DATABASE command to manage control files, including adding new ones, dropping existing ones, and backing them up.

Redo Log Files

Redo log files are an essential component of Oracle Database's recovery mechanism. They record all changes made to the database, ensuring data integrity and recoverability in case of failure.

Purpose of Redo Log Files

- Protect database changes against system failures
- Allow database recovery in case of crashes or power failures
- Support **ROLL FORWARD** operation during media recovery

Key Characteristics of Redo Log Files

- They store **all transactional changes** before they are written to the database.
- They operate in a **circular manner**, where Oracle reuses log files after switching.
- They support **data recovery** by rolling forward committed changes after a failure.

Types of Redo Log Files

1. **Online Redo Log Files** – Actively used by the database to record changes.
2. **Archived Redo Log Files** – Saved copies of redo logs used for recovery and backup.

Managing Tablespaces and Data Files in Oracle

In Oracle, **tablespaces** are logical storage units that help organize and manage database objects like tables and indexes. Each tablespace consists of **one or more data files**, which store the actual data on disk.

Types of Tablespaces

1. **System Tablespaces**
    - SYSTEM: Stores data dictionary and core database metadata.
    - SYSAUX: Stores additional Oracle components (e.g., AWR, Statspack).
2. **User Tablespaces**
    - Created for storing application data (e.g., USERS, DATA_TBS).
3. **Temporary Tablespaces**
    - Used for sorting and temporary operations (e.g., TEMP).
4. **Undo Tablespace**
    - Manages undo information for transactions (e.g., UNDOTBS1).

Storage Structure and Relationships in Oracle

Oracle Database uses a hierarchical storage structure to organize and manage data efficiently. This structure consists of logical and physical storage components that work together to store and retrieve data.

Storage Structure Overview

Oracle's storage structure is divided into:

(a) Logical Storage (How data is organized logically)

- **Database → Tablespaces → Segments → Extents → Blocks**
- These components define how data is stored and accessed.

(b) Physical Storage (How data is stored on disk)

- **Data Files** → Store actual data.
- **Redo Log Files** → Record transaction changes for recovery.
- **Control Files** → Maintain database metadata.

Logical Storage Structure

(a) Database

The highest-level logical storage unit that contains tablespaces, users, and schema objects.

(b) Tablespace

A tablespace is a **logical container** for storing database objects like tables and indexes.

- Each tablespace consists of **one or more data files** stored physically on disk.
- Types of tablespaces:
    - SYSTEM and SYSAUX (Oracle metadata)

- USER (application data)
- TEMP (temporary operations)
- UNDO (manages undo transactions)

Relationship:

- **One database** → contains **multiple tablespaces**
- **One tablespace** → consists of **one or more data files**

(c) Segments

A **segment** is a collection of extents used for storing database objects like:

- **Table Segments** → Stores table data.
- **Index Segments** → Stores index data.
- **Undo Segments** → Manages undo information.
- **Temporary Segments** → Used for sorting operations.

Relationship:

- **One tablespace** → contains **multiple segments**
- **One segment** → consists of **multiple extents**

(d) Extents

An **extent** is a contiguous group of Oracle database blocks allocated to a segment.

Relationship:

- **One segment** → consists of **multiple extents**
- **One extent** → consists of **multiple blocks**

(e) Data Blocks (Oracle Blocks)

The **smallest storage unit** in an Oracle database.

- The size of a block (e.g., 8KB, 16KB) is defined at database creation.
- It contains:
  - **Header** → Metadata of the block.
  - **Free Space** → Unused space.
  - **Row Data** → Actual stored data.

Relationship:

- **One extent** → consists of **multiple blocks**
- **One block** → stores **multiple rows**

---

3. Physical Storage Structure

(a) Data Files

- Store actual user and system data.
- Each tablespace has one or more data files.
- Example:

SELECT FILE_NAME, TABLESPACE_NAME FROM DBA_DATA_FILES;

(b) Control Files

- Maintain database metadata (e.g., database name, log file locations).
- Used during database startup.

(c) Redo Log Files

- Store changes made to the database for recovery purposes.

4. Relationships Summary

Storage Level	**Contains**
Database	Multiple Tablespaces
Tablespace	Multiple Segments
Segment	Multiple Extents
Extent	Multiple Blocks
Block	Multiple Rows

Managing Rollback Segments

A Rollback Segment in Oracle is a special type of segment used to store undo information for transactions. It ensures data consistency and enables rollback of uncommitted changes. Rollback segments were primarily used in manual undo management mode before Oracle introduced Automatic Undo Management (AUM) with UNDO tablespaces in Oracle 9i and later.

What is a Rollback Segment?

A Rollback Segment in Oracle stores undo information to:

- Roll back uncommitted transactions.
- Provide read consistency to queries.
- Support database recovery after failures.

Purpose of Rollback Segments

Rollback segments are used for:

✓ **Rolling back uncommitted transactions** in case of user errors.

✓ **Ensuring read consistency**, so queries see a consistent snapshot of data.

✓ **Recovering transactions** after an unexpected database failure.

Types of Rollback Segments

1. System Rollback Segment

- Automatically created in the SYSTEM tablespace.
- Required for maintaining database consistency.

2. User Rollback Segments

- Created by DBAs in a separate tablespace.
- Used for managing undo information for user transactions.

Managing Tables in Oracle

In Oracle, a table is a fundamental database object used to store structured data. Managing tables involves creating, modifying, deleting, and optimizing them to ensure efficiency and integrity.

1. Creating a Table

A table is created using the CREATE TABLE statement.

Basic Table Creation

```
CREATE TABLE employees (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100),
    salary NUMBER(10,2),
    department_id NUMBER,
    hire_date DATE DEFAULT SYSDATE
);
```

- emp_id → Primary Key (unique identifier).
- emp_name → Stores employee names.
- salary → Numeric field with 10 digits, 2 decimal places.
- department_id → Foreign key reference possible.
- hire_date → Defaults to the current date (SYSDATE).

---

2. Viewing Table Information

Check Existing Tables

```
SELECT TABLE_NAME FROM USER_TABLES;
```

Describe Table Structure

```
DESC employees;
```

or

```
SELECT COLUMN_NAME, DATA_TYPE, DATA_LENGTH FROM USER_TAB_COLUMNS
WHERE TABLE_NAME = 'EMPLOYEES';
```

---

3. Modifying a Table

(a) Adding a New Column

```
ALTER TABLE employees ADD (email VARCHAR2(100));
```

(b) Modifying a Column

Change the datatype or increase column size:

ALTER TABLE employees MODIFY (salary NUMBER(12,2));

(c) Dropping a Column

ALTER TABLE employees DROP COLUMN email;

4. Renaming a Table or Column

(a) Renaming a Table

ALTER TABLE employees RENAME TO emp_data;

(b) Renaming a Column

ALTER TABLE employees RENAME COLUMN emp_name TO full_name;

5. Managing Constraints

(a) Adding a Primary Key

ALTER TABLE employees ADD CONSTRAINT emp_pk PRIMARY KEY (emp_id);

(b) Adding a Foreign Key

ALTER TABLE employees ADD CONSTRAINT dept_fk FOREIGN KEY (department_id)
REFERENCES departments(department_id);

(c) Dropping a Constraint

ALTER TABLE employees DROP CONSTRAINT dept_fk;

6. Deleting Table Data

(a) Delete Specific Records

DELETE FROM employees WHERE emp_id = 101;

COMMIT;

(b) Truncate Table (Removes All Rows, No Rollback)

TRUNCATE TABLE employees;

(c) Drop a Table (Removes Structure & Data)

DROP TABLE employees;

What is an Index?

An **index** in Oracle is a database object that improves the speed of data retrieval operations on a table. It acts like a **lookup mechanism**, allowing queries to find rows quickly without scanning the entire table.

Key Benefits of Indexes

✓ Faster query performance (especially on large tables).

✓ Optimized filtering and sorting operations.

✓ Reduces I/O operations during data retrieval.

---

2. Types of Indexes in Oracle

(a) B-Tree Index (Default Index)

- The most commonly used index type.
- Maintains a **balanced tree structure** for quick lookups.

- Best for **highly selective queries** (queries that return a few rows).

(b) Unique Index

- Enforces **unique** values in a column.
- Automatically created when a **PRIMARY KEY** or **UNIQUE constraint** is defined.

Examples:

```
CREATE TABLE employees (
   emp_id NUMBER PRIMARY KEY,  -- Creates a Unique Index automatically
   emp_name VARCHAR2(100)
);
```

1. What is Data Integrity?

Data Integrity in Oracle ensures the accuracy, consistency, and reliability of data in a database. It prevents invalid, inconsistent, or duplicate data by enforcing rules and constraints on tables.

Types of Data Integrity:

1. Entity Integrity → Ensures each row has a unique identifier (Primary Key).
2. Domain Integrity → Ensures values fall within valid ranges (Data Types, Constraints).
3. Referential Integrity → Ensures relationships between tables remain valid (Foreign Keys).
4. User-Defined Integrity → Enforces business rules via Triggers, Views, and Procedures.

(a) PRIMARY KEY (Ensures Uniqueness)

- A PRIMARY KEY ensures each row has a unique, non-null identifier.
- Only one primary key per table, but it can consist of multiple columns (Composite Key).

```
CREATE TABLE employees (
   emp_id NUMBER PRIMARY KEY,
   emp_name VARCHAR2(100),
   salary NUMBER(10,2)
);
```

(b) UNIQUE (Prevents Duplicate Values)

- Ensures that a column (or a group of columns) has **unique values**.
- Allows **NULL values**.

```
CREATE TABLE employees (
   emp_id NUMBER PRIMARY KEY,
   emp_email VARCHAR2(100) UNIQUE
);
```

(c) NOT NULL (Prevents Missing Values)

- Ensures a column **cannot contain NULL values**.

```
CREATE TABLE employees (
   emp_id NUMBER PRIMARY KEY,
```

```
    emp_name VARCHAR2(100) NOT NULL
);
```

(d) CHECK (Restricts Values Based on a Condition)

- Ensures that column values satisfy a specific condition.

```
CREATE TABLE employees (
    emp_id NUMBER PRIMARY KEY,
    salary NUMBER(10,2) CHECK (salary > 0)
);
```

(e) FOREIGN KEY (Ensures Referential Integrity)

- Ensures a column's values exist in another table.
- Prevents **orphan records** (child records without a matching parent).

```
CREATE TABLE departments (
    department_id NUMBER PRIMARY KEY,
    department_name VARCHAR2(100)
);

CREATE TABLE employees (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100),
    department_id NUMBER,
    CONSTRAINT dept_fk FOREIGN KEY (department_id) REFERENCES
departments(department_id)
);
```

Managing Password Security and Resources in Oracle

Oracle provides various features to enhance password security and manage user resources to prevent unauthorized access and excessive resource consumption.

2. Managing User Passwords

(a) Creating a User with a Secure Password

```
CREATE USER john IDENTIFIED BY StrongP@ssw0rd;
```

- IDENTIFIED BY → Specifies the user's password.
- Use **strong passwords** with a mix of **uppercase, lowercase, numbers, and special characters**.

(a) User Authentication

Oracle requires each user to be **authenticated** using a password. Users can be:

- **Database authenticated** (stored in Oracle).
- **Externally authenticated** (via OS or LDAP).
- **Globally authenticated** (via Enterprise Directory Services).

(b) Password Policies and Profiles

To enforce password security, Oracle uses **Profiles**, which define policies such as:

✓ Password expiration and reuse policies.

✓ Locking accounts after multiple failed login attempts.

✓ Enforcing password complexity requirements.

Managing Users in Oracle

Oracle provides a robust User Management system to control access, authentication, and privileges in a database. Managing users involves creating, modifying, and removing users, along with assigning roles, privileges, and resource limits.

1. Creating Users in Oracle

A user in Oracle is an account that has access to database objects like tables, views, and procedures.

(a) Create a New User

```
CREATE USER john IDENTIFIED BY StrongP@ssw0rd;
```

- IDENTIFIED BY → Sets the user's **password**.
- By default, a new user **cannot** log in until privileges are granted.

(b) Creating a User with a Default Tablespace

```
CREATE USER john IDENTIFIED BY StrongP@ssw0rd
DEFAULT TABLESPACE users
TEMPORARY TABLESPACE temp
QUOTA 50M ON users;
```

2. Granting and Revoking Privileges

Oracle has **two types** of privileges:

✅ **System Privileges** → Allow users to perform database operations (e.g., CREATE TABLE, ALTER USER).

✅ **Object Privileges** → Allow users to **access and manipulate specific objects** (e.g., SELECT, INSERT on tables).

(a) Granting System Privileges

```
GRANT CREATE SESSION TO john;
```

(b) Granting Object Privileges

```
GRANT SELECT, INSERT ON employees TO john;
```

- john can **view and insert data** into the employees table.

(c) Revoking Privileges

To remove granted permissions:

```
REVOKE INSERT ON employees FROM john;
```

- john can no longer insert data into employees.

3. Managing User Roles

A **role** is a collection of privileges that can be assigned to multiple users.

(a) Creating a Role

```
CREATE ROLE manager_role;
```

(b) Granting Privileges to a Role

```
GRANT CREATE TABLE, CREATE VIEW TO manager_role;
```

(c) Assigning a Role to a User

```
GRANT manager_role TO john;
```

- Now john has all privileges assigned to manager_role.

(d) Revoking a Role from a User

```
REVOKE manager_role FROM john;
```

Privilege and Roles in DBMS

Confidentiality, integrity, and availability are the stamps of database security. Authorization is the allowance to the user or process to access the set of objects. The type of access granted can be any like, read-only, read, and write. Privilege means different Data Manipulation Language(DML) operations which can be performed by the user on data like INSERT, UPDATE, SELECT and DELETE, etc.

There are two methods by which access control is performed is done by using the following.

1. Privileges
2. Roles

Let’s discuss one by one.

Privileges :

The authority or permission to access a named object as advised manner, for example, permission to access a table. Privileges can permit permitting a particular user to connect to the database. In other words, privileges are the allowance to the database by the database object.

- **Database privileges —** A privilege is permission to execute one particular type of SQL statement or access a second persons' object. Database privilege controls the use of computing resources. Database privilege does not apply to the Database administrator of the database.

- **System privileges —** A system privilege is the right to perform an activity on a specific type of object. for example, the privilege to delete rows of any table in a database is system privilege. There are a total of 60 different system privileges. System privileges allow users to CREATE, ALTER, or DROP the database objects.

- **Object privilege —** An object privilege is a privilege to perform a specific action on a particular table, function, or package. For example, the right to delete rows from a table is an object privilege. For example, let us consider a row of table GEEKSFORGEEKS that contains the name of the employee who is no longer a part of the organization, then deleting that row is considered as an object privilege. Object privilege allows the user to INSERT, DELETE, UPDATE, or SELECT the data in the database object

Following are the differences between system privileges and object privileges.

Sr. No	System privileges	Object privileges
1.	This privileges is normally granted by a Database Administrative to users.	This privileges are granted by the owner of the object.
2.	This privileges are used to prevent or permit DDL statements such as create View, Table, session etc.	This privileges are used to prevent or permit DML statements such as Select, Insert, Update and Delete etc.
3.	This privileges allow the users to manage databases and servers.	This privileges allows users to perform certain action upon database objects.

4.	Syntax: Grant privileges to Username;	Syntax: Grant privileges ON object TO username;

(a) Granting System Privileges
GRANT CREATE SESSION TO john;
(b) Granting Object Privileges
GRANT SELECT, INSERT ON employees TO john;

- john can **view and insert data** into the employees table.

(c) Revoking Privileges
To remove granted permissions:
REVOKE INSERT ON employees FROM john;

- john can no longer insert data into employees.

Roles :

A role is a mechanism that can be used to allow authorization. A person or a group of people can be allowed a role or group of roles. By many roles, the head can manage access privileges very easily. The roles are provided by the database management system for easy and managed or controlled privilege management.